

Motion Detection in Image Sequences Using Spatial and Temporal Filtering

Kabhilesh Giri

M.S. Electrical and Computer Engineering
Northeastern University
giri.k@northeastern.edu

Aidan Syrgak Uulu

M.S. Electrical and Computer Engineering
Northeastern University
syrgakuulu.a@northeastern.edu

Abstract

This project presents a modular motion detection system based on temporal image filtering techniques combined with spatial smoothing and thresholding. The objective is to analyze how different filtering strategies influence motion extraction quality in image sequences. Four spatial smoothing configurations were implemented: no smoothing, 3×3 box filtering, 5×5 box filtering, and Gaussian smoothing with adjustable sigma. Three temporal derivative operators were evaluated: a first-order 1-D differential operator, a centered difference filter $0.5[-1, 0, 1]$, and a 1-D derivative of Gaussian (DoG). Motion responses were converted into binary motion masks using both manual thresholding and adaptive thresholding methods. The system was tested on multiple real-world datasets. Results were analyzed visually across grayscale conversion, smoothing output, temporal derivative response, threshold mask, and final masked overlay. Experimental observations highlight the trade-off between noise suppression and motion sensitivity, demonstrating how parameter selection in spatial and temporal filtering directly impacts motion detection robustness.

Description of Algorithms

The implemented system follows a structured video-processing pipeline in which each stage applies a clearly defined algorithmic transformation to the input frames. The overall workflow consists of frame acquisition, spatial smoothing, temporal

derivative computation, thresholding, and visualization. Each stage is modular and driven by configuration parameters selected through the graphical interface, allowing different algorithmic strategies to be instantiated dynamically. The pipeline begins with the Class “ImageSequenceReader”, which is responsible for loading the input data and converting the frames into grayscale images. This component reads an ordered sequence of image frames from a specified directory and returns two synchronized outputs: the original BGR frames and their grayscale counterparts. Grayscale conversion is performed immediately after loading, since all subsequent processing stages operate on single-channel intensity images. Maintaining consistent frame dimensions and ordering is essential at this stage, as all downstream algorithms assume spatial and temporal alignment of the data.

Following frame acquisition, the system performs spatial smoothing to reduce high-frequency noise prior to temporal analysis. Two primary filtering approaches are available. The first option employs a “BoxFilter” with either a 3×3 or 5×5 kernel. This filter replaces each pixel value with the average of its local neighborhood using mean filtering. Border handling is implemented through reflection to avoid artificial edge artifacts. While the 3×3 configuration provides moderate smoothing, the 5×5 kernel applies stronger averaging at the cost of increased blurring of fine motion details. You can also choose to select None to skip smoothing.

The second smoothing option utilizes a “Gaussian2DFilter”, parameterized by a user-defined standard deviation (σ). The kernel size is derived from σ to ensure sufficient spatial coverage, and Gaussian convolution is applied using symmetric border reflection. Compared to uniform averaging, Gaussian smoothing better preserves structural edges while attenuating random noise. In both cases, smoothing is applied independently to each frame, producing a temporally consistent sequence prepared for motion analysis. After spatial preprocessing, the pipeline computes a temporal derivative to estimate motion between frames. Multiple strategies are supported, each differing in complexity and robustness. The easiest approach is the Class “SimpleTemporalDifference”, which operates on two consecutive frames and computes the absolute pixel-wise difference. This method highlights intensity changes directly but remains highly sensitive to noise and illumination variation. A more stable alternative is the Class “OneDCenteredDifference”, which uses three consecutive frames and applies a symmetric temporal operator of the form $0.5(\text{next} - \text{previous})$, or as described, $0.5[-1, 0, 1]$. By using information from both past and future frames relative to the current index, this centered scheme reduces temporal jitter and produces smoother motion estimates. The output is converted to absolute magnitude and clipped to the valid intensity range for further processing.

For higher robustness, the system implements a Class “One D Derivative of Gaussian” (temporal DoG) method. This approach constructs a temporal window whose size depends on the selected σ parameter and applies a first-order Gaussian derivative filter along the time axis. The filtered stack produces a temporally smoothed derivative, from which the center-aligned frame is selected. Absolute values and clipping ensure compatibility with later stages. Compared to simple differencing, this method sacrifices immediate responsiveness in exchange for improved noise suppression and stability.

Temporal processing is coordinated by the “Temporal_Derivative” wrapper, which manages frame buffering and output alignment. A deque structure maintains a sliding window whose length matches the selected derivative strategy. Once sufficient frames are accumulated, the chosen computation is executed, and the resulting derivative frame is aligned to the appropriate temporal index. For odd window sizes, alignment corresponds to the center frame; for even sizes, alignment follows the most recent frame. This abstraction ensures consistent indexing regardless of the derivative method employed.

The next stage applies thresholding to convert derivative magnitudes into binary motion masks. Two strategies are provided. The Class “Manual Threshold” method applies a fixed global threshold value: pixels exceeding the threshold are classified as motion (255), while others are set to zero. This approach offers predictability and interpretability but requires careful tuning for each sequence. Alternatively, the Class “Adaptive Threshold” method determines threshold values based on local image statistics. Using a Gaussian-weight neighborhood, the algorithm computes a local mean and subtracts a constant offset before classifying each pixel. This localized decision rule allows the system to adapt to spatial illumination variations and heterogeneous motion intensity. Both strategies are encapsulated within a Threshold wrapper that standardizes mask generation and maintains alignment with the derivative outputs.

Finally, the visualization stage produces interpretable outputs. A binary mask is combined with the original BGR frame using a bitwise AND operation, preserving only those pixels identified as motion while suppressing static regions. The system also supports synchronized playback of intermediate stages, including original frames, grayscale images, smoothed outputs, derivatives, masks, and overlays. This structured visualization enables direct comparison of algorithmic effects at each step of the pipeline.

Basically, the implemented framework integrates classical spatial filtering, temporal differentiation, and thresholding techniques into a cohesive and configurable motion-detection pipeline. Each algorithmic component fulfills a distinct role, and the modular design allows controlled experimentation with different smoothing scales, derivative operators, and segmentation strategies. The result is a flexible system capable of illustrating how algorithmic choices influence motion estimation and visual interpretation.

Experiments

Following the development of these algorithms, a multitude of experiments were run. In Table I, you can find the the list of some of the runs with specified values of parameters used. As mentioned before, the $0.5[-1,0,1]$ filter is referred to as “centered”. “DoG” refers to 1D Derivative of a Gaussian, with a standard deviation denoted as “ σ ”. Note, that the threshold in the below experiments was held constant at 10, as to not pollute the analysis from varying the other parameters. The manual and adaptive thresholds were tested after. The experiments were as follows:

Table I
Experiments and Values of Parameters Used

#	Spatial Smoothing	Temporal Derivative
1	None	Centered
2	None	DoG $\sigma = 0.5$
3	None	DoG $\sigma = 1.0$
4	None	DoG $\sigma = 2.0$
5	3x3 Box	Centered
6	5x5 Box	Centered
7	2D Gaussian $\sigma = 0.5$	Centered
8	2D Gaussian $\sigma = 1.0$	Centered
9	2D Gaussian $\sigma = 2.0$	Centered
10	2D Gaussian $\sigma = 0.5$	DoG $\sigma = 0.5$
11	2D Gaussian $\sigma = 1.0$	DoG $\sigma = 0.5$
12	2D Gaussian $\sigma = 2.0$	DoG $\sigma = 0.5$
13	2D Gaussian $\sigma = 0.5$	DoG $\sigma = 1.0$
14	2D Gaussian $\sigma = 1.0$	DoG $\sigma = 1.0$
15	2D Gaussian $\sigma = 2.0$	DoG $\sigma = 1.0$
16	2D Gaussian $\sigma = 0.5$	DoG $\sigma = 2.0$

17	2D Gaussian $\sigma = 1.0$	DoG $\sigma = 2.0$
18	2D Gaussian $\sigma = 2.0$	DoG $\sigma = 2.0$

Once completed, we decided on the best performing “run” by looking over the images and comparing them visually. After deciding on the best configuration, we run another experiment on that configuration with the adaptive threshold strategy described in the previous section.

Observations

To show what our experiments actually produced, we decided to focus on one consistent frame (**#216**), from the “EnterExitCrossingPaths2cor” dataset. We picked this frame on purpose. It captures the exact moment when two people cross paths, which makes it a difficult detection case. If an algorithm struggles anywhere, it’s probably going to struggle here.

Each output figure below has 6 images - summaries of the motion d-detection pipeline for a single run and the frame: the “original” is the raw color frame, “grayscale” is the same frame reduced to one channel for processing, “smoothing” shows the spatially filtered grayscale frame (to reduce noise before temporal processing), “1D derivative” is the temporal -derivative response (bright pixels are rapid intensity changes over time, motion), “threshold mask” is an image derived from that response where white/bright pixels are classified as motion and black pixels are background, and the “masked” / “overlay” image applies that binary mask to the original frame so only moving regions remain visible. In short, you’re seeing the progression from raw data to noise r-reduced input to motion signal to motion/no motion-motion classification and finally, isolated motion regions.

We start with Experiment #1: no spatial smoothing and the simple centered temporal derivative (Figure 1). This is essentially the “raw” version of the system. Unsurprisingly, the output looks noisy. There’s a lot of background activity that shouldn’t really be there, and the masks feel

jittery and unstable. It does detect motion, but it's messy.

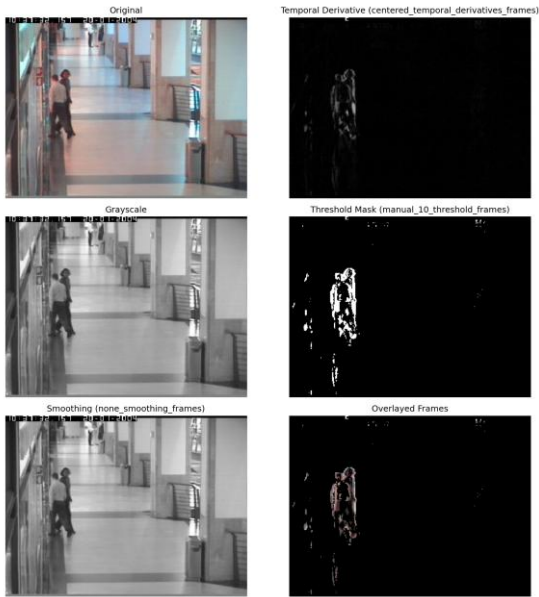


Figure 1. Experiment #1, no spatial smoothing, centered temporal derivative

In Experiment #2 (Figure 2), we keep spatial smoothing off but switch the temporal derivative to DoG with $\sigma = 0.5$. Even this small change makes a visible difference. Some of the background pixels disappear, and the output looks a bit more stable. The improvement isn't dramatic, but it's noticeable. The centered filter reacts too aggressively to frame-to-frame fluctuations, while DoG smooths across time and calms things down.

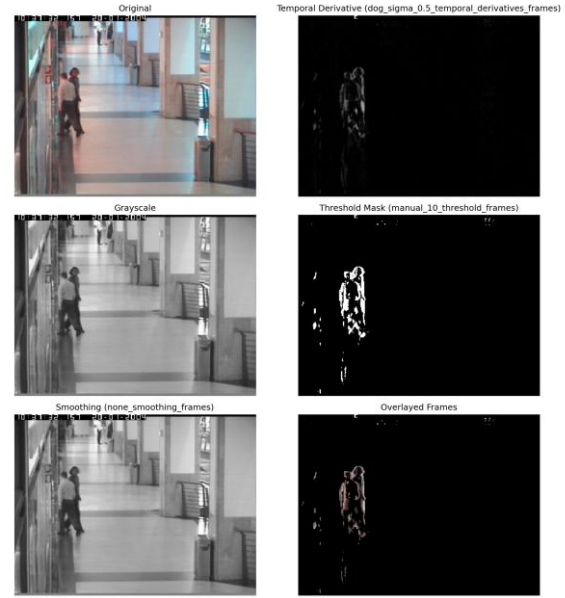


Figure 2. Experiment #2, no spatial smoothing, DoG $\sigma = 0.5$

When we push the temporal sigma further to 2.0 in Experiment #4 (Figure 3), the effect becomes much stronger. A lot of background patterns and noise fade away. The tradeoff, though, is clear: the outlines of the moving people are less sharp. Because the temporal filter now spans a wider window, the motion gets “spread out” across frames. So yes, it's cleaner, but also blurrier.

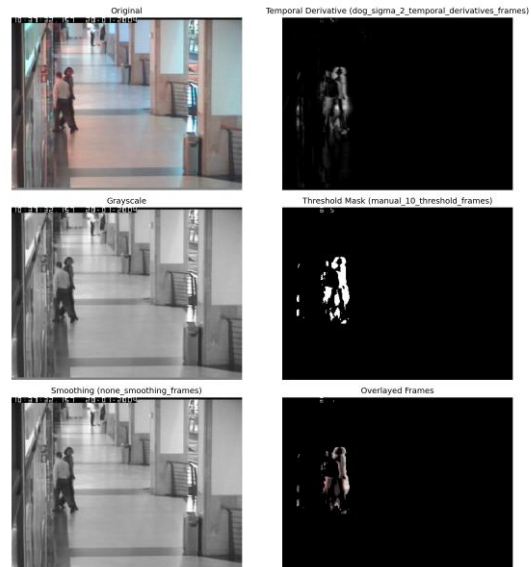


Figure 3. Experiment #4, no spatial smoothing, DoG $\sigma = 2.0$

After exploring temporal filtering alone, we moved on to spatial smoothing, starting with the 3×3 box filter (Experiment #5, Figure 4). The first thing you notice is the smoothing panel; it's clearly blurred. That blur carries over into the motion outputs as well. The jitter decreases a bit, but the images start to lose crispness.

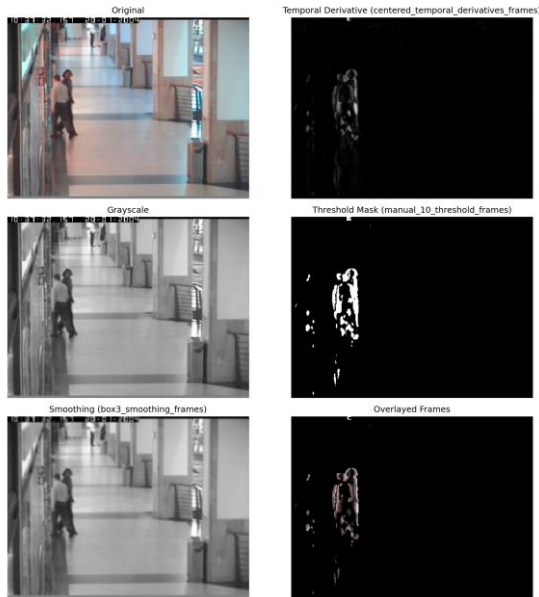


Figure 4. Experiment #5, 3×3 box spatial smoothing, centered temporal derivative

With the 5×5 box filter (Experiment #6, Figure 5), the blur becomes even more pronounced. Noise and jitter are mostly gone, but now the figures look like blobs. The shapes are over-smoothed and slightly convoluted. It almost feels like we removed too much structure along with the noise.

This behavior highlights an important structural weakness of uniform averaging filters. The box filter does not discriminate between noise and meaningful edges; it treats all local intensity variations equally. As a result, while random high-frequency noise is reduced, genuine motion boundaries are also softened. In motion detection, they directly influence how confidently the temporal derivative responds. When edges become diluted, the derivative magnitude spreads across neighboring pixels, creating thicker but less precise motion regions. The mask may appear cleaner, but

the spatial localization of motion becomes less accurate.

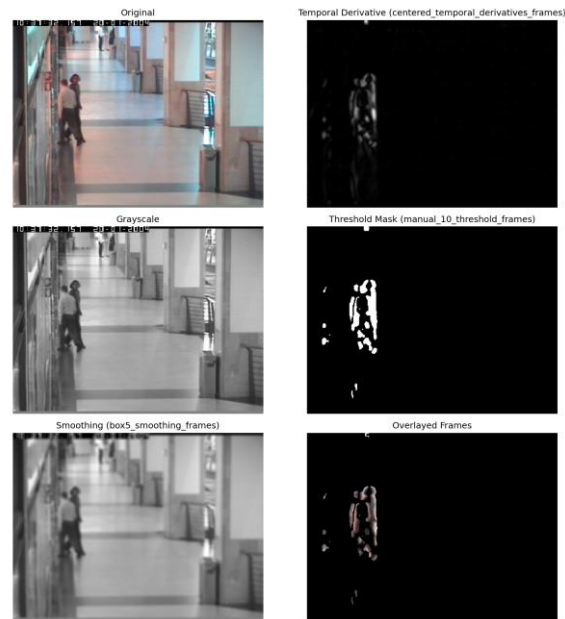


Figure 5. Experiment #6, 5×5 box spatial smoothing, centered temporal derivative

Next came the 2D Gaussian filter. Although we tested way more Gaussian combinations, we'll focus on the more representative ones. We chose DoG of $\sigma = 1$ as a middle ground, and because it provided solid results. In Experiment #13 (Figure 6), we set the spatial Gaussian σ to 0.5 and used DoG with $\sigma = 1$. Visually, it might remind you of the box filters at first glance, but there's an important difference. The background doesn't look blocky anymore. Instead of fake-looking motion artifacts, it resembles more natural noise. The key difference with the Gaussian filter is how it treats neighboring pixels. Unlike the box filter, which gives equal weight to every pixel in the window, the Gaussian filter gives more importance to pixels closer to the center. Because of this weighted averaging, edges are softened more gently instead of being flattened uniformly. The result is smoothing that feels more controlled and less artificial.

In Experiment #13, with spatial Gaussian $\sigma = 0.5$ and temporal DoG $\sigma = 1$, the motion regions appear smoother without completely losing their

shape. The silhouettes of the two people are still recognizable. Their outlines are not as sharp as the no-smoothing case, but they are not distorted into blobs either. The derivative image looks more stable, and the threshold mask appears more consistent across the moving regions.

These results suggest that Gaussian filtering offers a better trade-off between smoothing and detail preservation. It reduces unwanted variations but avoids the heavy blur introduced by uniform averaging.

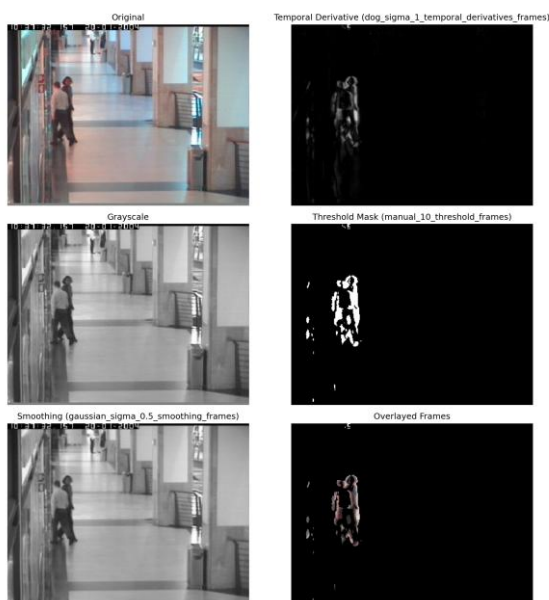


Figure 6. Experiment #13, 2D gaussian $\sigma = 0.5$, DoG $\sigma = 1$

When we increase the spatial Gaussian σ to 1.0 in Experiment #14 (Figure 7), things improve even further. Background noise drops noticeably, but the motion of the two crossing figures remains clear. This configuration felt like a sweet spot, not too blurry, not too jittery. It balances spatial and temporal smoothing without letting either dominate.

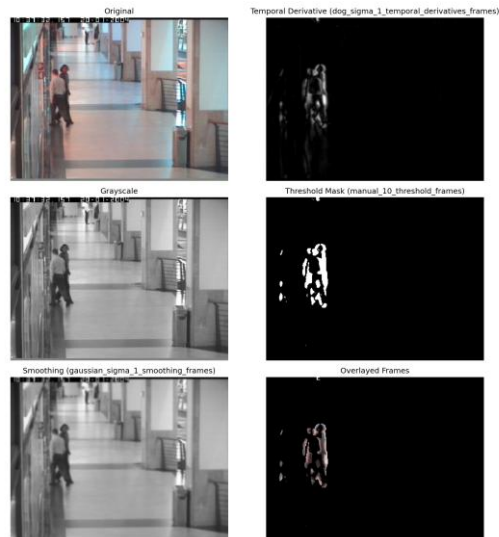


Figure 7. Experiment #14, 2D Gaussian $\sigma = 1$, DoG $\sigma = 1$

That said, the configuration we found most effective overall was Experiment #12 (Figure 8): spatial Gaussian $\sigma = 2.0$ combined with temporal DoG $\sigma = 0.5$. Here, heavy spatial smoothing removes pixel-level noise and background artifacts almost entirely. At the same time, the lighter temporal smoothing preserves motion detail. The two figures are easier to distinguish, and their shapes appear more solid and well-defined. The masks look cleaner and more confident.

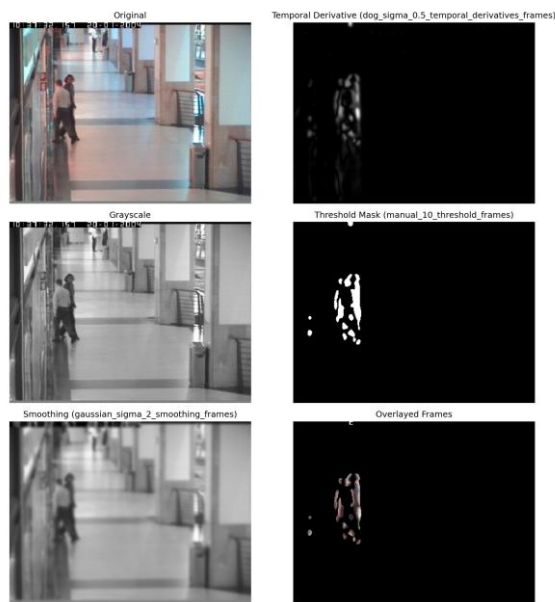


Figure 8. Experiment #12, 2D Gaussian $\sigma = 2$, DoG $\sigma = 0.5$

Out of curiosity, we also tested the extreme case in Experiment #18 (Figure 9), where both spatial and temporal sigma were set to 2.0. As expected, noise and jitter nearly vanished, but so did important motion details. The figures blur across space and time, and the person walking away from the camera is barely visible. It's a clear example of over-smoothing. When both domains are heavily filtered, meaningful structure dissolves together with the noise.



Figure 9. Experiment # 18, 2D Gaussian $\sigma = 2$, DoG $\sigma = 2$

As mentioned before, we then chose what we thought was our best experiment, #8, and applied the previously mentioned “AdaptiveThreshold” method. It uses local neighborhood statistics to compute a different threshold for each pixel rather than applying a single global value across the entire frame. This leads to being very good at detecting outlines but losing track of the whole shape of moving objects and introducing a lot of noise and jitter.

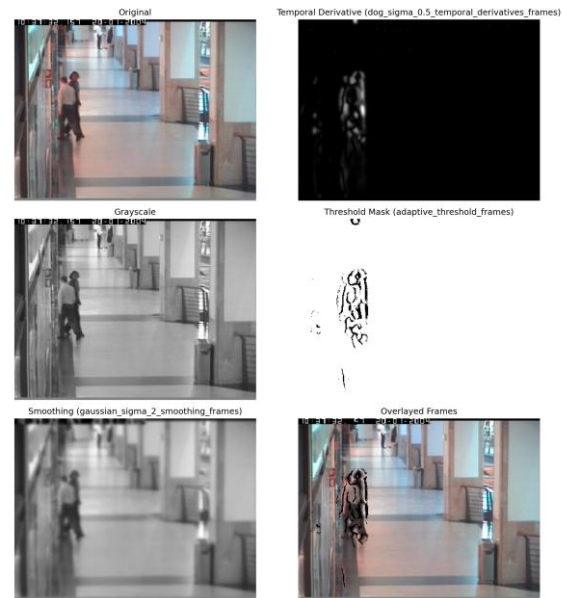


Figure 10. Adaptive threshold

Conclusions

The experiments conducted throughout this project reveal a clear pattern: the interaction between spatial and temporal filtering ultimately determines the quality of motion detection.

The comparison between the centered difference operator and the temporal derivative of Gaussian (DoG) makes this especially evident. The simple $0.5[-1, 0, 1]$ filter, while mathematically straightforward, is excessively sensitive to noise. Background fluctuations are big, jitter becomes visible, and masks exhibit unstable behavior. Replacing this operator with a DoG immediately stabilizes the output. Even at low temporal sigma values, the DoG reduces background artifacts and produces a cleaner derivative image.

However, increasing temporal sigma introduces a tradeoff. Larger values suppress noise and remove unwanted background patterns, but they also widen the temporal window. As a result, motion boundaries become blurred, and sharp transitions dissolve. At $\sigma = 2.0$, much of the background noise disappears, yet edge sharpness suffers. The system becomes stable, but at the expense of responsiveness and structural precision. This teaches an important lesson: temporal

smoothing must be applied carefully, as excessive filtering pollutes the motion information the system is meant to capture.

The spatial domain has its own lessons. Box filters, both 3×3 and 5×5 , consistently introduce artificial patterns. Their uniform averaging generates blocky textures and, in many cases, fake motion artifacts in static backgrounds. The 5×5 configuration increases this effect, producing over-smoothed, blob-like shapes that distort object boundaries. In contrast, 2D Gaussian spatial filtering yields significantly cleaner backgrounds. Noise retains a more texture rather and motion structures are preserved better. The advantage of Gaussian filtering becomes visually apparent across nearly all experiments.

The most successful configurations demonstrate that spatial and temporal filtering cannot be treated as independent decisions. A balanced configuration, such as 2D Gaussian smoothing with $\sigma = 1.0$ combined with DoG at $\sigma = 1.0$, achieves a stable equilibrium: the output is neither overly blurry nor excessively jittery. Yet the strongest overall performance in our case came from a more asymmetric strategy. Using stronger spatial smoothing at $\sigma = 2.0$ together with lighter temporal smoothing at $\sigma = 0.5$ complements each other well. High spatial filtering eliminates pixel-level sensor noise before temporal processing occurs, allowing the temporal derivative to operate on a cleaner signal. Because the signal is already stabilized spatially, the temporal component requires less aggressive smoothing. The result are masks that are clean, differentiated, and structurally solid.

The case of excessive smoothing in both domains further shows this interaction. When both spatial and temporal sigma values are set high at 2.0, blurring occurs across space and time simultaneously. Motion edges dissolve, structures disappear, and figures become undetectable. This over-smoothing demonstrates that filtering domains are kind of interdependent: aggressive smoothing in one domain allows for lighter filtering in the other,

but applying both too much results in loss of information.

All in all, there is no universal parameter set that guarantees best performance. The best configuration depends on motion speed, camera noise characteristics, and scene complexity. Nevertheless, an important finding here is that: asymmetric strategies, heavy in one domain and light in the other, can outperform symmetric ones. Especially in challenging scenarios, such as this one with intersecting motion paths and complex frame interactions. In these cases, experimenting with spatial and temporal parameters outside of just “picking the middle ground” is important.

References

1. We compare temporal-derivative motion masks to a classical background subtraction baseline (Stauffer & Grimson, 1999).
2. For quantitative evaluation of motion masks, we follow benchmark practices from ChangeDetection.net (Goyette et al., 2012).
3. Prompting followed OpenAI’s recommended practices (OpenAI, n.d.), emphasizing specificity and iterative refinement.
4. As a classical motion-estimation baseline, optical flow can be computed using the Horn–Schunck formulation (Horn & Schunck, 1981).
5. We use a standardized change-detection benchmark to evaluate mask quality (Goyette et al., 2012).